

Day 63: Weekly Revision — Developer Basics

1. Day Number

Day 63

2. Topic Name

Revision of Days 61–62: Developer Basics

Today you will revise:

- `pip`
- virtual environments with `venv`
- project folders
- dependencies
- `requirements.txt`

3. Connection to Previous Days

On **Day 61**, you learned how Python projects can install external packages using `pip` and how a **virtual environment** keeps those packages isolated.

On **Day 62**, you learned how to organize a small Python project using files such as `main.py`, folders such as `data/`, and a `requirements.txt` file.

Today combines both ideas into one small developer workflow:

Create project → create virtual environment → activate it → install dependency → create Python code → save dependencies.

4. Revision Summary of Days 61–62

Day 61 — `pip` and Virtual Environments

`pip` is Python's package installer.

For example:

```
pip install requests
```

This installs the `requests` package.

A virtual environment creates an isolated place for packages used by one project.

Create one with:

```
python -m venv venv
```

Activate it before installing project packages.

Day 62 — Project Structure and requirements.txt

A small Python project might look like:

```
my_project/
|
├── main.py
├── requirements.txt
└── data/
```

main.py contains your Python code.

data/ can contain files your program uses.

requirements.txt records the external packages your project needs.

For example:

```
requests==2.x.x
```

The exact version depends on the version installed on your computer.

5. Important Topics

pip

Used to install Python packages.

```
pip install package_name
```

venv

Used to create an isolated Python environment.

```
python -m venv venv
```

Project folder

The main folder containing everything related to your project.

Example:

```
weather_project/
```

Dependencies

Dependencies are external packages your project needs.

For example, if your Python code uses:

```
import requests
```

then requests is a dependency.

requirements.txt

A text file containing the project's dependencies.

It helps another developer install the same packages.

6. Foundational Notes

Think of a Python project as having two main parts.

Your own Python code

You write this yourself.

Example:

```
print("Hello from my project!")
```

This could be stored inside:

```
main.py
```

External dependencies

These are packages written by other developers that your program uses.

For example:

```
import requests
```

You normally install them using:

```
pip install requests
```

A virtual environment keeps these installed packages associated with your project instead of mixing everything together globally.

A typical beginner workflow is:

```
Create project folder
↓
Create virtual environment
↓
Activate virtual environment
↓
```

```
Install package
  ↓
Write Python code
  ↓
Save dependencies
```

7. Easy Example

Imagine you are creating this project:

```
hello_project/
|
├── main.py
├── requirements.txt
└── venv/
```

You create the environment:

```
python -m venv venv
```

Activate it.

On Windows:

```
venv\Scripts\activate
```

On macOS/Linux:

```
source venv/bin/activate
```

Then install a small package:

```
pip install requests
```

Your `main.py` could contain something simple such as:

```
import requests

print("My project is ready!")
```

The important idea is not what requests does yet.

The important idea is:

```
main.py
  ↓ uses
requests
```

```
↓ installed with  
pip  
↓ inside  
virtual environment
```

8. Revision Problem Statement

Create a **very small Python project**.

Your project should:

1. Have its own project folder.
2. Have a virtual environment.
3. Activate the virtual environment.
4. Install **one external package**.
5. Contain a `main.py` file.
6. Contain a `requirements.txt` file.
7. Save the installed dependency inside `requirements.txt`.

Keep the program extremely simple.

For example, your Python program can simply import the installed package and print:

```
Project setup complete!
```

The main goal is to practice **project setup**, not complicated Python programming.

9. Concepts Used

This exercise practices:

- command-line basics
 - project organization
 - pip
 - packages
 - dependencies
 - venv
 - activating a virtual environment
 - Python files
 - `requirements.txt`
 - pip freeze
-

10. Thought Process

Before typing commands, think through the project in this order.

First: Where will my project live?

You need one folder for the project.

Second: Does this project need an isolated environment?

Yes, so create a virtual environment.

Third: Is the environment active?

Check this before installing the package.

Fourth: What external package will the project use?

Choose one simple package such as requests.

Fifth: Where does my Python code go?

Put it in `main.py`.

Sixth: How will another developer know which package to install?

Record the dependency in `requirements.txt`.

The mental model is:

```
Project
├── My code
│   └── main.py
├── Dependencies
│   └── requirements.txt
└── Isolated Python environment
    └── venv/
```

11. Pseudocode-Style Setup Steps

Do not treat this as the finished solution. Use it as a roadmap.

```
START

Create a new project folder

Move into the project folder

Create a virtual environment

Activate the virtual environment

Check that the environment is active
```

Install one simple package with pip

Create main.py

Inside main.py:
import the installed package
print a simple message

Create requirements.txt

Save the installed dependency into requirements.txt

Run main.py

Check that there are no errors

END

12. Suggested Solving Approach — Simple Developer Workflow

Work one step at a time.

Step 1: Create the project

Start with an empty folder.

Step 2: Create the environment

Use:

```
python -m venv venv
```

Step 3: Activate it

Make sure the environment is active **before** installing anything.

Step 4: Install one package

Use `pip install`.

Don't install many packages for this exercise.

Step 5: Create your Python program

Create:

```
main.py
```

Keep the program to only a few lines.

Step 6: Record the dependency

Remember that `pip` installing a package does **not automatically mean you have documented it for the project**.

The dependency should also be recorded in:

```
requirements.txt
```

Step 7: Test the project

Run:

```
python main.py
```

If it works without an import error, your basic setup is working.

13. Easy Edge Cases

Environment is not active

You might install the package outside your project's virtual environment.

For example, you run:

```
pip install requests
```

without activating `venv`.

Your project may then behave differently when the virtual environment is activated.

Check: Is your virtual environment active before installing packages?

Missing `requirements.txt`

Your program may run perfectly on your machine, but another developer will not immediately know which external packages it needs.

Make sure the project contains:

```
requirements.txt
```

Empty `requirements.txt`

Creating the file alone is not enough.

If your project depends on `requests`, that dependency should appear in the file.

14. Common Mistakes to Avoid

1. Installing packages before activating the virtual environment

Activate the environment first.

2. Putting Python code inside `requirements.txt`

`requirements.txt` is for dependency information, not Python code.

3. Writing package names inside `main.py` instead of importing them

Python dependencies are normally used with statements such as:

```
import requests
```

4. Forgetting to create `main.py`

Your project should contain a clear Python entry file for this exercise.

5. Thinking `venv` and `requirements.txt` are the same thing

They solve different problems:

```
venv
= isolated environment

requirements.txt
= list of project dependencies
```

6. Installing lots of packages

This exercise only needs **one** external package.

15. Quick Self-Check Questions

Question 1

What does `pip` mainly do?

Think about: external Python packages.

Question 2

Why do we use a virtual environment?

Think about: keeping packages for different projects separate.

Question 3

What is a dependency?

Think about: code your project needs but that you did not write yourself.

Question 4

What is the purpose of `requirements.txt`?

Think about: helping someone recreate the project's package setup.

Question 5

Which should usually happen first?

```
pip install requests
```

or

```
activate the virtual environment
```

Explain why.

16. Hint Only

For your revision exercise, aim for a final structure similar to:

```
small_project/  
├── main.py  
├── requirements.txt  
└── venv/
```

Remember this order:

```
folder  
→ venv  
→ activate  
→ pip install  
→ main.py  
→ requirements.txt  
→ run program
```

One useful command from the previous lessons can inspect installed dependencies and redirect them into a file.

Think about:

```
pip ____ > requirements.txt
```

Try to remember the missing command from **Day 61–62** rather than looking at the completed solution.

Day 63 Goal

By the end of today's revision, you should understand that a small real Python project is more than just a .py file:

```
Python code
+ isolated environment
+ installed dependencies
+ dependency record
= basic Python project workflow
```

Don't build anything bigger today. The purpose of Day 63 is to make the developer workflow from Days 61–62 feel familiar.